

Separation of Concerns in Apex

FFLib Series • Session 001

Andrew Fawcett

Code With Sally

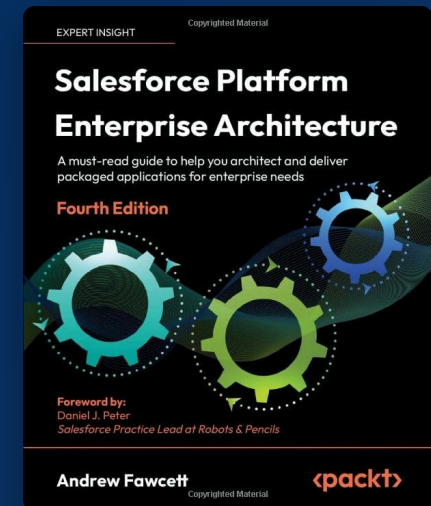


About me

Independent Salesforce Consultant | Former CPO,
Salesforce, Heroku | CTO, FinancialForce.com

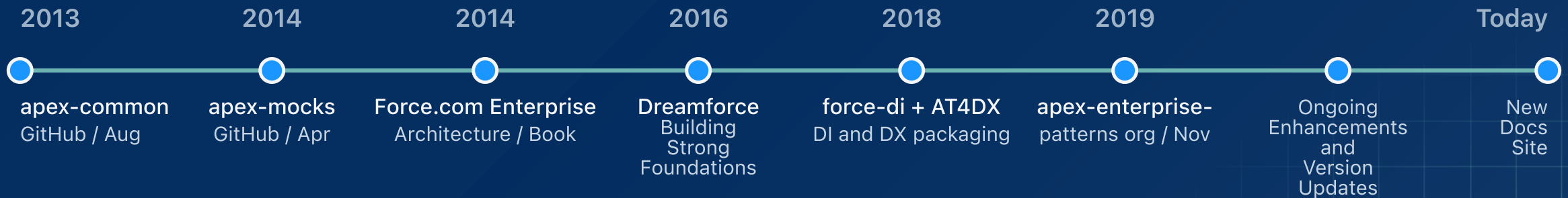
- **Independent consultant and architect** — Salesforce platform development, PaaS integrations, and agentic AI
- **Open source and guidance** — creator of DLRS, Apex frameworks, and *Salesforce Platform Enterprise Architecture*
- **Salesforce Customers and Partners Advisory** — architecture and scale, AI-enhanced workflows, org health and codebase mentoring, and ISV product advisory

Reduce complexity, unblock engineering, and leave architectures — and teams — able to evolve.



fflib – a brief history

- **Early ISV (2009)** – FinancialForce.com (ff); enterprise heritage (C, Java)
- **Motivation** – repeatable structure for large Apex codebases
- `fflib-apex-common` – Service, Domain, Selector, Unit of Work
- `fflib-apex-mocks` – Stub API for isolated tests
- **Adoption** – used by many larger enterprise orgs, partner SIs and ISVs
- **Today** – Launching brand new documentation site



fflib — core contributors



John M. Daniel

Senior Director of Digital
Platforms
Steampunk, Inc.

♥♥ Jeeps



John Storey

Staff Software Engineer
Thrivent

♥♥ General Aviation



David Esposito

SVP Engineering
Leap Event Technology

♥♥ Taking things apart



Andrew Fawcett

CEO
AndyInTheCloud Consulting Ltd

♥♥ Lego and retro computing



Matt Gerry

Salesforce CTA
Coding With The Force

♥♥ Retro computers and
consoles

fflib — what's evolved

- **Domains** — can be just behavior / task / operation methods
- **Sidecar trigger handlers** — fine for object logic related to CRUD
- **Combined domains** — still fine if they do both
- **Services** — now prefer instance methods over static
- **Selectors** — don't need every field; just the common ones
- **Feature selectors** — can be feature-specific, not only object-centric
- **Application class** — optional; mocking can be done without it - [blog](#)
- **Apex interfaces** — now recommended only for dependency injection
- **fflib-apex-mocks** — no longer a dependency



What is wrong with this picture?

What's wrong with this picture?

Developer “A” — creates a controller

OpportunityCreateInvoiceController.cls

```
1 public inherited sharing class OpportunityCreateInvoiceController {
2     @AuraEnabled(cacheable=false)
3     public static Id createInvoice(
4         Id opportunityId, Decimal discountPercentage) {
5         try {
6             Opportunity opportunity = [
7                 SELECT Id, AccountId, CloseDate, Description,
8                 (SELECT Id, UnitPrice, Quantity,
9                 PricebookEntry.Product2Id, ...
10                FROM OpportunityLineItems)
11            FROM Opportunity WHERE Id = :opportunityId
12        ];
13
14        // optional discount logic ...
15        Invoice__c invoice = new Invoice__c();
16        // set invoice fields ...
17        insert invoice;
18        for (OpportunityLineItem line : opportunity.OpportunityLineItems)
19            // build invoice lines ...
20        }
21        insert allLines;
22        return invoice.Id;
23    } catch (Exception e) {
24        throw new AuraHandledException(e.getMessage());
25    }
26 }
27 }
```

Developer “B” — later creates a batch job

CreateInvoicesJob.cls

```
1 public inherited sharing class CreateInvoicesJob
2     implements Database.Batchable<SObject>, ... {
3     private List<Exception> exceptions = new List<Exception>();
4
5     public void execute(
6         Database.BatchableContext context,
7         List<Opportunity> opps) {
8         for (Opportunity opp : opps) {
9             try {
10                 OpportunityCreateInvoiceController
11                     .createInvoice(opp.Id, 0);
12             } catch (Exception e) {
13                 exceptions.add(e);
14             }
15         }
16     }
17
18     public Database.QueryLocator start(...) { /* ... */ }
19     public void finish(...) { /* ... */ }
20 }
```

Code samples are reduced. Missing variables are not what's wrong. Samples assume user mode, API 67.0.

So what was wrong with that picture?

Developer “A” – OpportunityCreateInvoiceController

OPPORTUNITYCREATEINVOICECONTROLLER ISSUE

Business Logic Misplaced Logic such as this should be client agnostic.

Error Handling Risk of partial updates as request will still commit.

Developer “B” – CreateInvoicesJob

CREATEINVOICESJOB ISSUE

Controller Logic Reuse Controller logic not written to be reused elsewhere.

Error Handling Exceptions being logged are UI exceptions.

Bulkification Indirect bulkification issue.

What's wrong with this picture?

■ opportunityApplyDiscount.js

```
1 import { getRecord, getFieldValue, updateRecord } from "lightning/uiRecordApi";
2 import { getRelatedListRecords } from "lightning/uiRelatedListApi";
3 import DISCOUNT_TYPE_FIELD from "@salesforce/schema/Opportunity.DiscountType__c";
4
5 export default class OpportunityApplyDiscount extends LightningElement {
6   @wire(getRecord, { recordId: "$recordId", fields: OPP_FIELDS })
7   wiredOpportunity;
8   @wire(getRelatedListRecords, {
9     parentRecordId: "$recordId", relatedListId: "OpportunityLineItems", ...
10  })
11  wiredLineItems;
12
13  async handleApply() {
14    const factor = 1 - (this.discountPercentage / 100);
15    const discountType = getFieldValue(this.wiredOpportunity.data, DISCOUNT_TYPE_FIELD);
16    const updates = [];
17
18    for (const line of this.wiredLineItems.data.records) {
19      const approved = ...Product2.DiscountingApproved__c;
20      if (discountType === "Approved Products" && !approved) continue;
21      updates.push({
22        fields: { Id: line.id, UnitPrice: line.fields.UnitPrice.value * factor }
23      });
24    }
25    await Promise.all(updates.map((r) => updateRecord(r)));
26  }
27 }
```

So what was wrong with that picture?

opportunityApplyDiscount.js

OPPORTUNITYAPPLYDISCOUNT

ISSUE

Business Logic Misplaced

Discount rules live in JavaScript — not client agnostic.

Reuse

Flow, REST, and Agent Actions cannot call this implementation.

Error Handling

`Promise.all` with `updateRecord` — partial updates may still commit.

What's wrong with this picture?

CreateInvoice.cls

```
1 public inherited sharing class CreateInvoice {
2     @InvocableMethod(label='Create Invoice', ...)
3     public static List<Result> execute(List<Request> requests) {
4         Set<Id> opportunityIds = ...; // from requests
5         Map<Id, Opportunity> opportunities = [ ... WHERE Id IN :opportunityIds ];
6         Map<Id, Invoice__c> invoicesByOpportunityId = new Map<Id, Invoice__c>();
7         Map<Id, List<InvoiceLine__c>> linesByOpportunityId = new Map<Id, List<InvoiceLine__c>>();
8         for (Request request : requests) {
9             Opportunity opportunity = opportunities.get(request.opportunityId);
10            Decimal discountFactor = ...; // from request.discountPercentage
11            invoicesByOpportunityId.put(request.opportunityId, new Invoice__c(...));
12            List<InvoiceLine__c> lines = new List<InvoiceLine__c>();
13            for (OpportunityLineItem lineItem : opportunity.OpportunityLineItems) {
14                lines.add(new InvoiceLine__c(UnitPrice__c = ... * discountFactor, ...));
15            }
16            linesByOpportunityId.put(request.opportunityId, lines);
17        }
18        insert invoicesByOpportunityId.values();
19        List<InvoiceLine__c> allLines = new List<InvoiceLine__c>();
20        for (Id oppId : linesByOpportunityId.keySet()) {
21            // wire each line to its invoice Id, add to allLines
22        }
23        insert allLines;
24        List<Result> results = ...; // compose results
25        return results;
26    }
27 }
```


So what was wrong with that picture?

CreateInvoice.cls

CREATEINVOICE

ISSUE

Business Logic Misplaced

Creation of the Invoice and apply discount is not the job of this class.

Complexity

Actual business logic is hidden due to query, transformation and DML logic.

Lack of SoC is not just a reuse problem

- 🤔 How would **you** expose “**apply discount**” and “**create invoice**” as an **API**?
- 🤔 How would **you** call it from a **React** app?
- 🤔 How would **you** wire it to an **Agent Action** or **Flow**?
- ⚠️ If you have this problem — your app is not ready to embrace change
- ⚠️ Duplication is not the real problem — **code base durability** is
- ❤️ Your business logic is **the** most important logic of your app

How durable is your code when change occurs?

How users interact with your application changes a lot — protect against it with SoC



🤔 Business Logic Impact

■ OpportunityController.cls ■ OpportunityInvocable.cls ■ InvoiceController.cls ■ InvoiceInvocable.cls

```
// logic copied — every new client type
VF · Aura · LWC · Agent Action · React · Next?
└─ OpportunityController.applyDiscount(...)
└─ OpportunityInvocable.applyDiscount(...)
└─ InvoiceController.createInvoice(...)
└─ InvoiceInvocable.createInvoice(...)
```



■ OpportunityService.cls ■ InvoiceService.cls

```
// same services — every client type
LWC · REST · Batch · Flow · Agent Action · Next?
└─ OpportunityService.applyDiscount(...)
└─ InvoiceService.createInvoice(...)
```

❤️ Your business logic is **the** most important logic in your app — and when it's everywhere, it becomes a **significant investment risk**

So what is “Separation of Concerns” then?

“ *Separation of concerns (SoC) is a design principle in computer science and software engineering; it holds that a complex problem should be divided into distinct concerns – aspects or issues – that can be analyzed, addressed or managed individually, even when they belong to the same system. This allows focusing on one issue at a time, reducing cognitive load and complexity.*

”

[Wikipedia, “Separation of Concerns”](#) (as at 2025)

So what is “DRY” then?

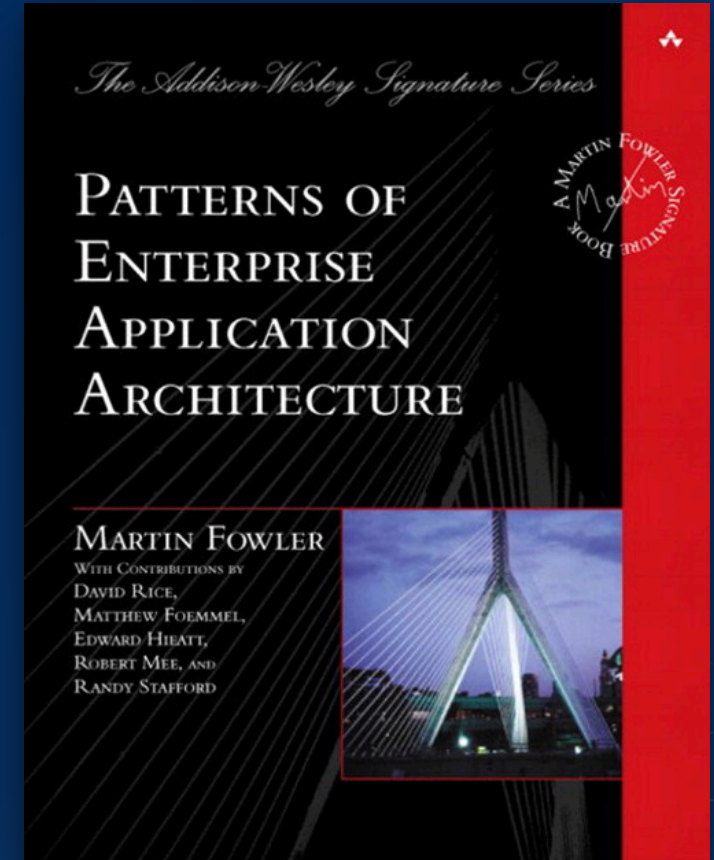
Don't Repeat Yourself

“ *Every piece of knowledge must have a single, unambiguous, authoritative representation within a system.* ”

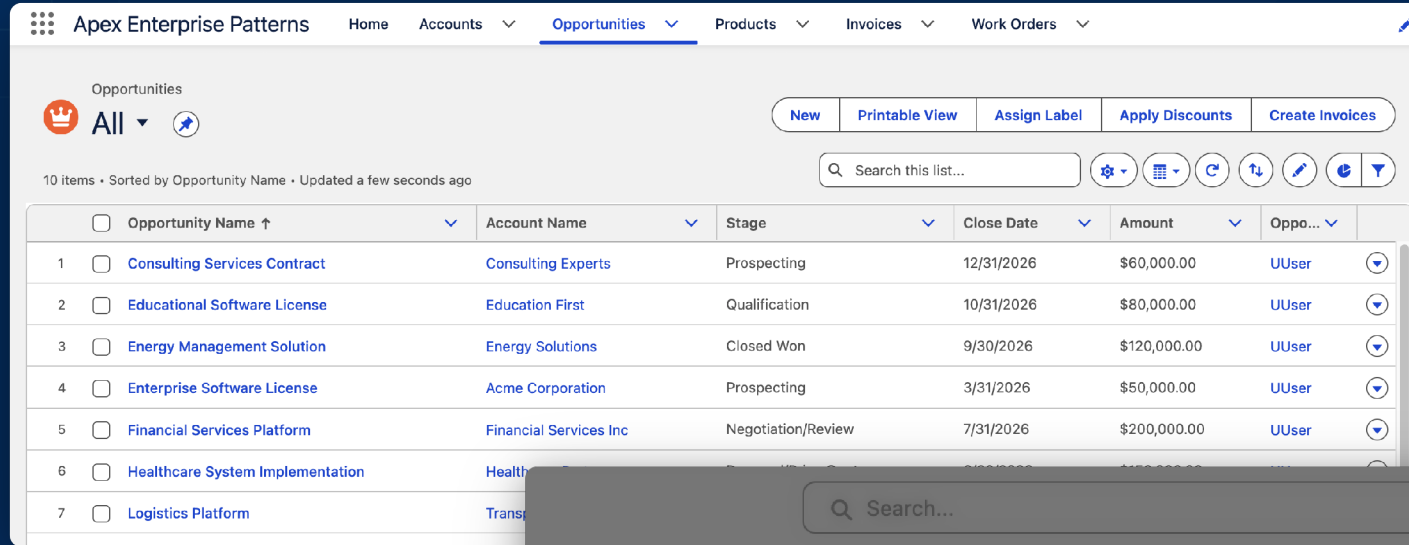
[Wikipedia, Don't repeat yourself \(DRY\).](#)

Design Patterns for SoC and Force.com

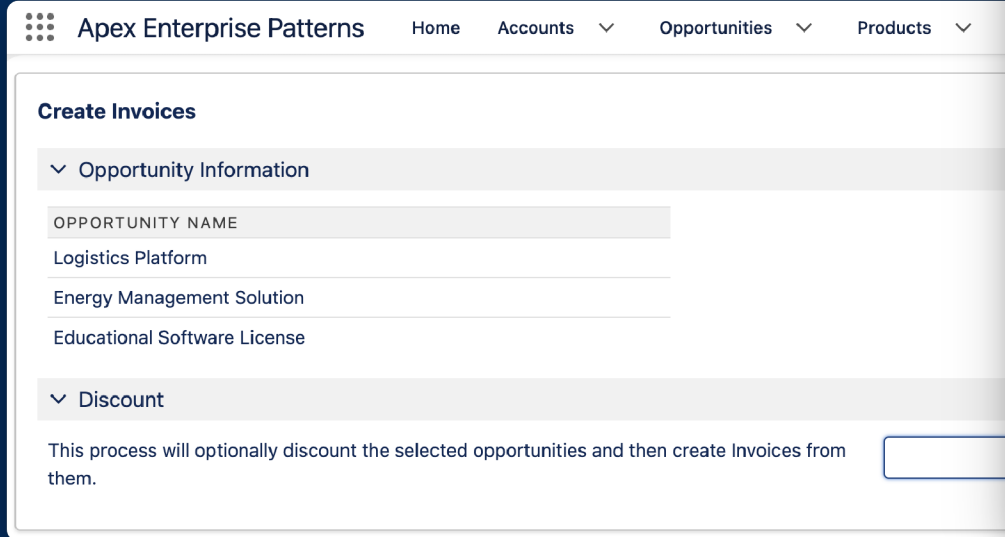
- **Service Layer** — one place for application operations
- **Domain Layer** — behaviour that is true of the object
(yet another wrapper / trigger pattern)
- **Selector Layer** — one place for SOQL
- Martin Fowler — [Patterns of Enterprise Application Architecture](#)



Sample Code - Buttons and Lightning UIs



	☐ Opportunity Name ↑	Account Name	Stage	Close Date	Amount	Opportunity User
1	☐ Consulting Services Contract	Consulting Experts	Prospecting	12/31/2026	\$60,000.00	UUser
2	☐ Educational Software License	Education First	Qualification	10/31/2026	\$80,000.00	UUser
3	☐ Energy Management Solution	Energy Solutions	Closed Won	9/30/2026	\$120,000.00	UUser
4	☐ Enterprise Software License	Acme Corporation	Prospecting	3/31/2026	\$50,000.00	UUser
5	☐ Financial Services Platform	Financial Services Inc	Negotiation/Review	7/31/2026	\$200,000.00	UUser
6	☐ Healthcare System Implementation	Healthcare System	Closed Won	10/31/2026	\$150,000.00	UUser
7	☐ Logistics Platform	Transportation	Closed Won	10/31/2026	\$100,000.00	UUser



Create Invoices

▼ Opportunity Information

OPPORTUNITY NAME

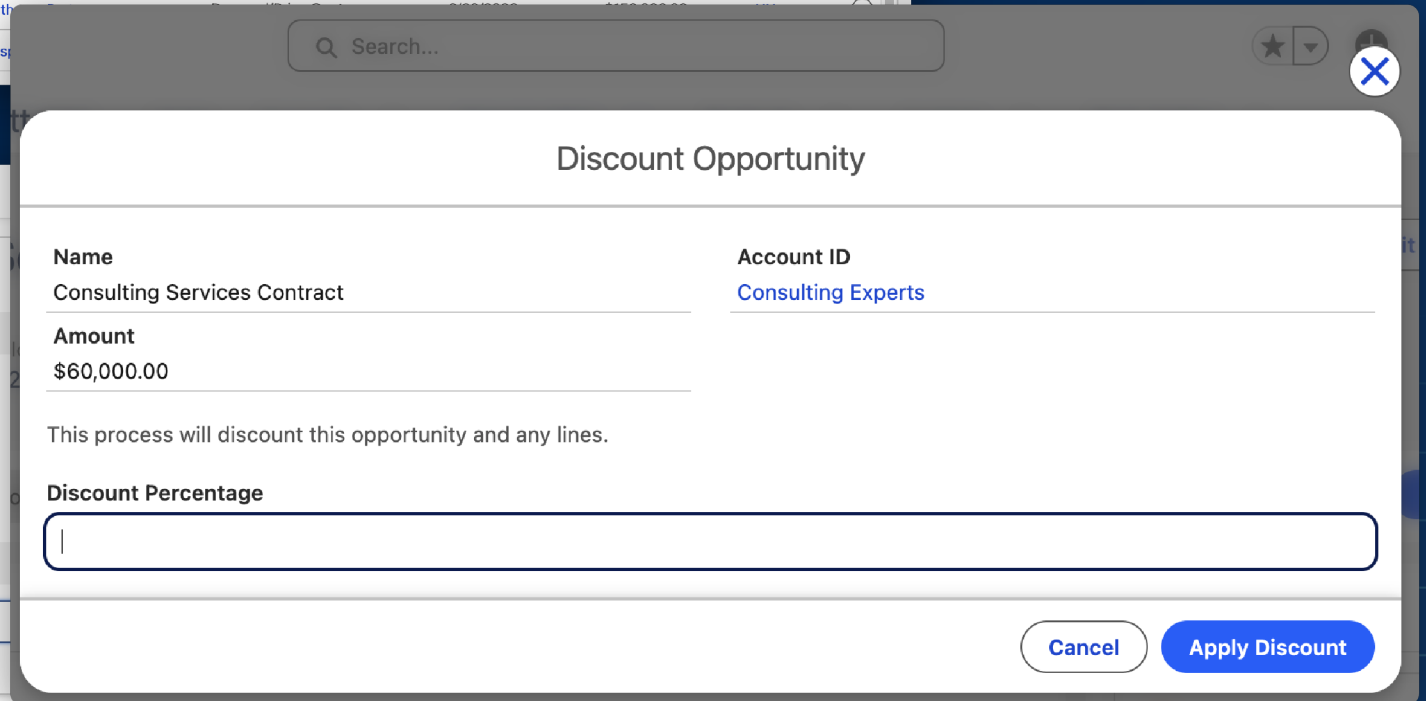
Logistics Platform

Energy Management Solution

Educational Software License

▼ Discount

This process will optionally discount the selected opportunities and then create Invoices from them.



Discount Opportunity

Search...

Name
Consulting Services Contract

Account ID
Consulting Experts

Amount
\$60,000.00

This process will discount this opportunity and any lines.

Discount Percentage

Cancel Apply Discount

Sample Code - REST and AgentForce

zsh

```
$ sf api request rest \
```

```
'/services/apexrest/opportunities/006Ru00000VQICN  
IA5/applydiscount?discount=10' \  
--method POST --include \  
--target-org session001-soc \  
--body '{}'
```

```
HTTP/1.1 200
```

Apex Enterprise Patterns

HomeAccountsOpportunitiesProductsInvoicesWork Orders

Opportunity

Agent Invoice 1787427688

Account Name

Close Date

Amount

Opportunity Owner

Agent Ops Account 1787427688

8/22/2026

\$1,000.00

User User

Prospecti...

Qualificati...

Needs An...

Value Pro...

Id. Decisi...

Perceptio...

Proposal/...

Negotia...

Activity

Details

Chatter

Opportunity Owner

Private

Opportunity Name

Account Name

Type

User User

Agent Invoice 1787427688

Agent Ops Account 1787427688

Prospecting

Amount

Expected Revenue

Close Date

Next Step

Stage

\$1,000.00

\$100.00

8/22/2026

Prospecting

Opportunity Operations

I can apply discounts to opportunities and create invoices from them. What would you like to do?

Apply 10% discount and create an invoice.

A 10% discount has been applied and the invoice has been created. The new Invoice Number is INV-00000001. Would you like to do anything else with this opportunity or another one?

Describe your task or ask a question...

Sample Code

force-app.without-soc / main / default

> applications

classes

- CreateInvoice.cls
- CreateInvoice.cls-meta.xml
- CreateInvoicesJob.cls
- CreateInvoicesJob.cls-meta.xml
- CreateInvoicesJobTest.cls
- CreateInvoicesJobTest.cls-meta.xml
- CreateInvoiceTest.cls
- CreateInvoiceTest.cls-meta.xml
- DeveloperWorkItemsTest.cls
- DeveloperWorkItemsTest.cls-meta.xml
- InvoicingService.cls
- InvoicingService.cls-meta.xml
- InvoicingServiceTest.cls
- InvoicingServiceTest.cls-meta.xml
- OpportunitiesResource.cls
- OpportunitiesResource.cls-meta.xml
- OpportunitiesTriggerTest.cls
- OpportunitiesTriggerTest.cls-meta.xml
- OpportunityApplyDiscountController.cls

Vanilla SoC

You don't need a library or tooling to follow SoC — so for now neither of these samples use fflib or other libraries. Later in this series we apply fflib over the SoC sample and highlight the value along the way.

force-app.with-soc

main

> applications

classes

- actions
- apis
- batchjobs
- controllers
 - OpportunityApplyDiscountController.cls
 - OpportunityApplyDiscountController.cls-meta.xml
- domains
- selectors
- services
 - InvoicingService.cls
 - InvoicingService.cls-meta.xml
 - OpportunitiesService.cls
 - OpportunitiesService.cls-meta.xml

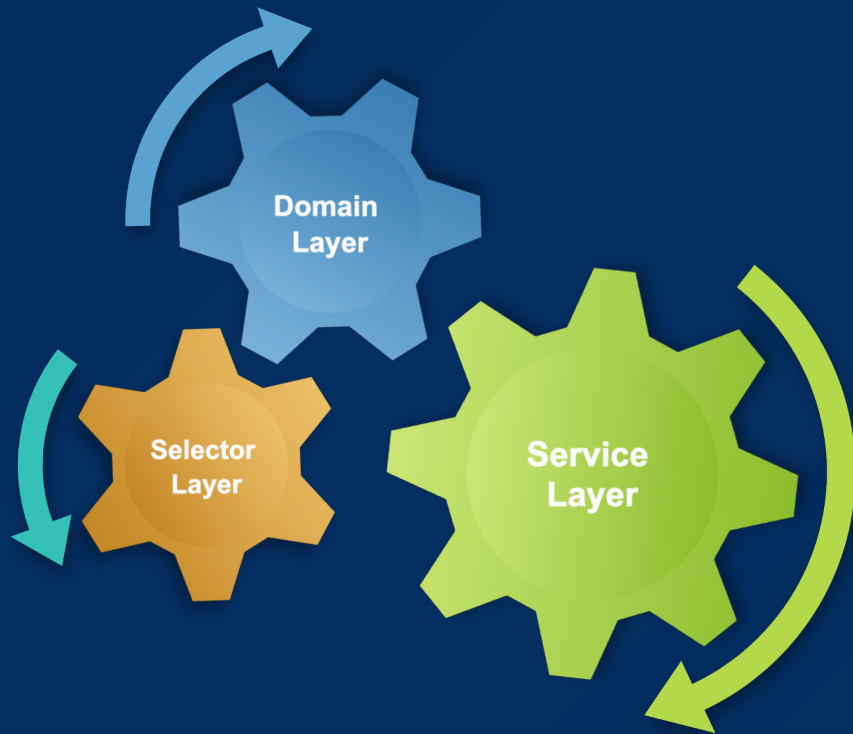
Without SOC

With SOC

Sample Code - Platform Features

- **Buttons** – Apply Discount, Create Invoice, Quick Opportunity...
- **Triggers** – defaults, validation, related Account activity
- **Batch** – Create Invoices job
- **Invocable** – Create Invoice (Flow / Agent Action)
- **Agentforce** – Apply discounts and create invoices
- **REST** – Opportunities resource

Pattern Checklist



PATTERN CHECKLIST

Separation of Concerns

☐

Service Layer

☐

Domain Layer

☐

Selector Layer

☐

Salesforce Platform SOC Layers

Presentation

- **You define:** Layouts, Custom Buttons, Flow, Record Types, Formula, Reports, Dashboards, Agent Builder
- **You code:** Apex Controllers, Visualforce, Aura, Lightning Components, React

Integration

- **You define:** Custom Objects, Custom Fields
- **You code:** Apex SOAP, Apex REST, Invocable Methods, Agent Actions, MCP, `global` Apex

Business Logic

- **You define:** Formula, Validation Rules, Workflow, Process Builder, Flow, Sharing Rules, Prompt Builder
- **You code:** Apex Services, DataWeave, Callouts

Data Access

- **You define:** Data Loaders, Permission Sets, Profiles
- **You code:** SOQL, SOSL and Salesforce APIs

Database

- **You define:** Custom Objects, Custom Fields, Relationships, Rollups, D360
- **You code:** Apex Triggers, External Data Sources (External Objects)



Suggested Class Folder Layout

Presentation

OpportunityCreateInvoiceController.cls OpportunityApplyDiscountController.cls OpportunityInvoicingCon
// /classes/controllers

Integration

ApplyDiscount.cls CreateInvoice.cls OpportunitiesResource.cls
// /classes/actions · /classes/apis



Business Logic

OpportunitiesService.cls InvoicingService.cls
// /classes/services

Opportunities.cls OpportunityLineItems.cls Accounts.cls ...
// /classes/domains

OpportunitiesTriggerHandler.cls
// /classes/triggerHandlers

Data Access

OpportunitiesSelector.cls OpportunityLineItemsSelector.cls AccountsSelector.cls ProductsSelector.c
// /classes/selectors

Database

OpportunitiesTriggerHandler.cls
// /classes/triggerHandlers

The Testing Triangle loves SOC

Faster tests,
more durable code base

With SOC

Slower tests, less coverage,
less durable to future change

Without SOC

More focused on
broad end-to-end given
investment in core
logic below

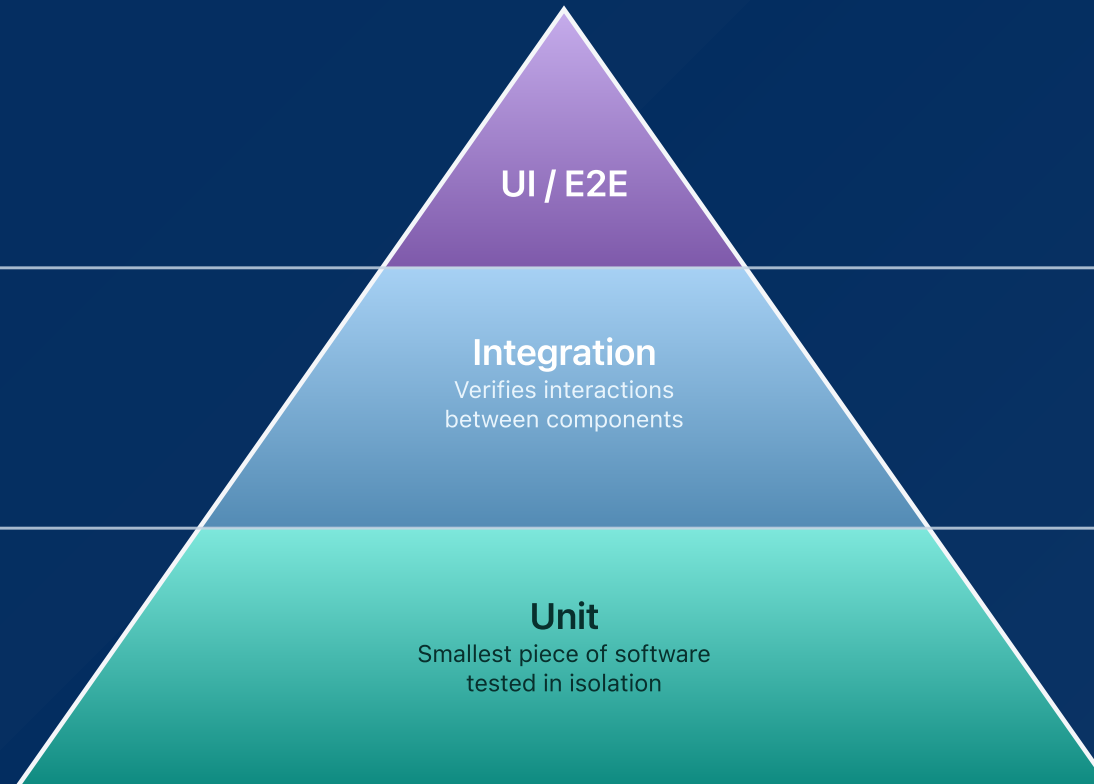
Focus can end up going
deeper than needed to
cover integration or
unit test gaps

Traditional DML tests
focus on testing services
perform end to end

Traditional DML
based tests are slower
and can detract from
authoring further tests

Leverage Apex Stub API
to mock services, domain,
selectors, handlers
for fast in-memory tests

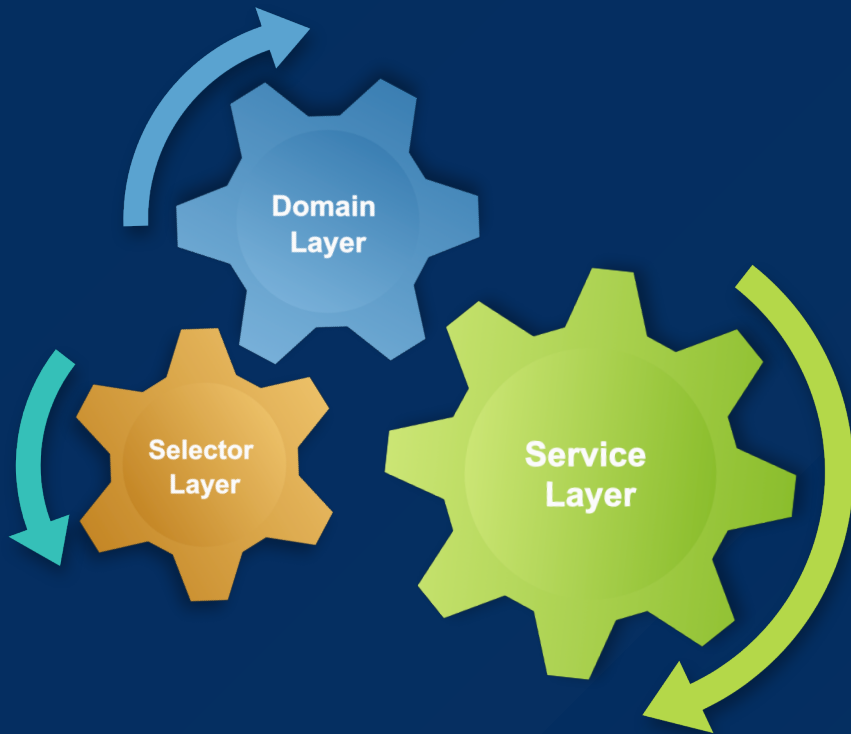
End up being utility
classes only, due to
lack of mock
injection points



“ *Separation of Concerns is a requirement to build well-factored, durable and lasting applications – regardless of fflib or other library support.*

”

Pattern Checklist



PATTERN CHECKLIST

Separation of Concerns



Service Layer



Domain Layer



Selector Layer

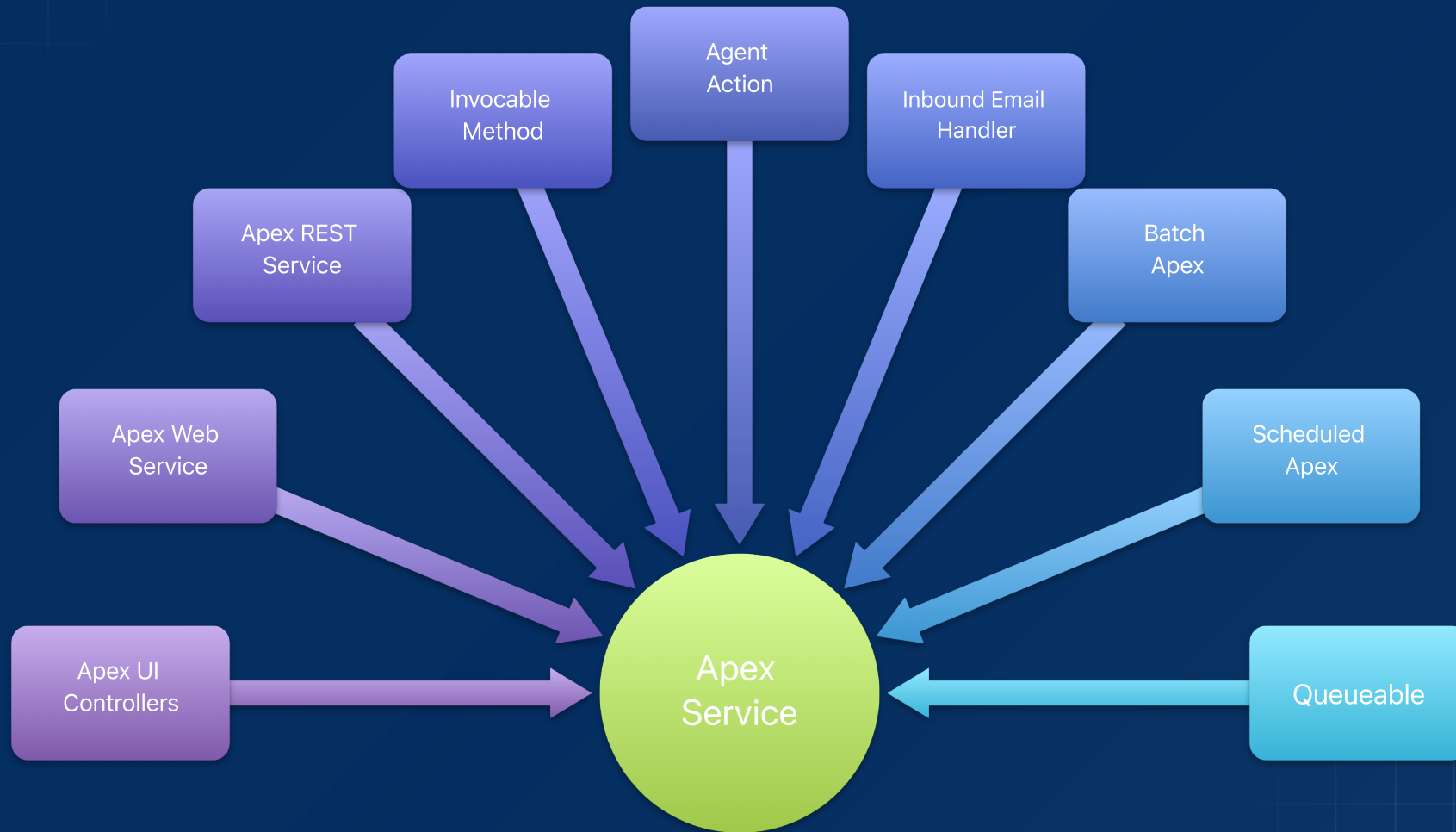


Service ➔ Task Orientated Logic

- A Service represents a **feature** of the application
 - Invoicing — `InvoicingService.cls`
 - Opportunity Discounting — `OpportunitiesService.cls` or `OpportunitiesDiscountService.cls`
- Each method is a **task** within that feature — `applyDiscounts`, `createInvoices`, or `submitJob`
- Think of it as the **conductor** — the band handles querying and object-specific logic; the Service sets the tempo and the rules of the orchestration



Services have many Consumers



Service Checklist

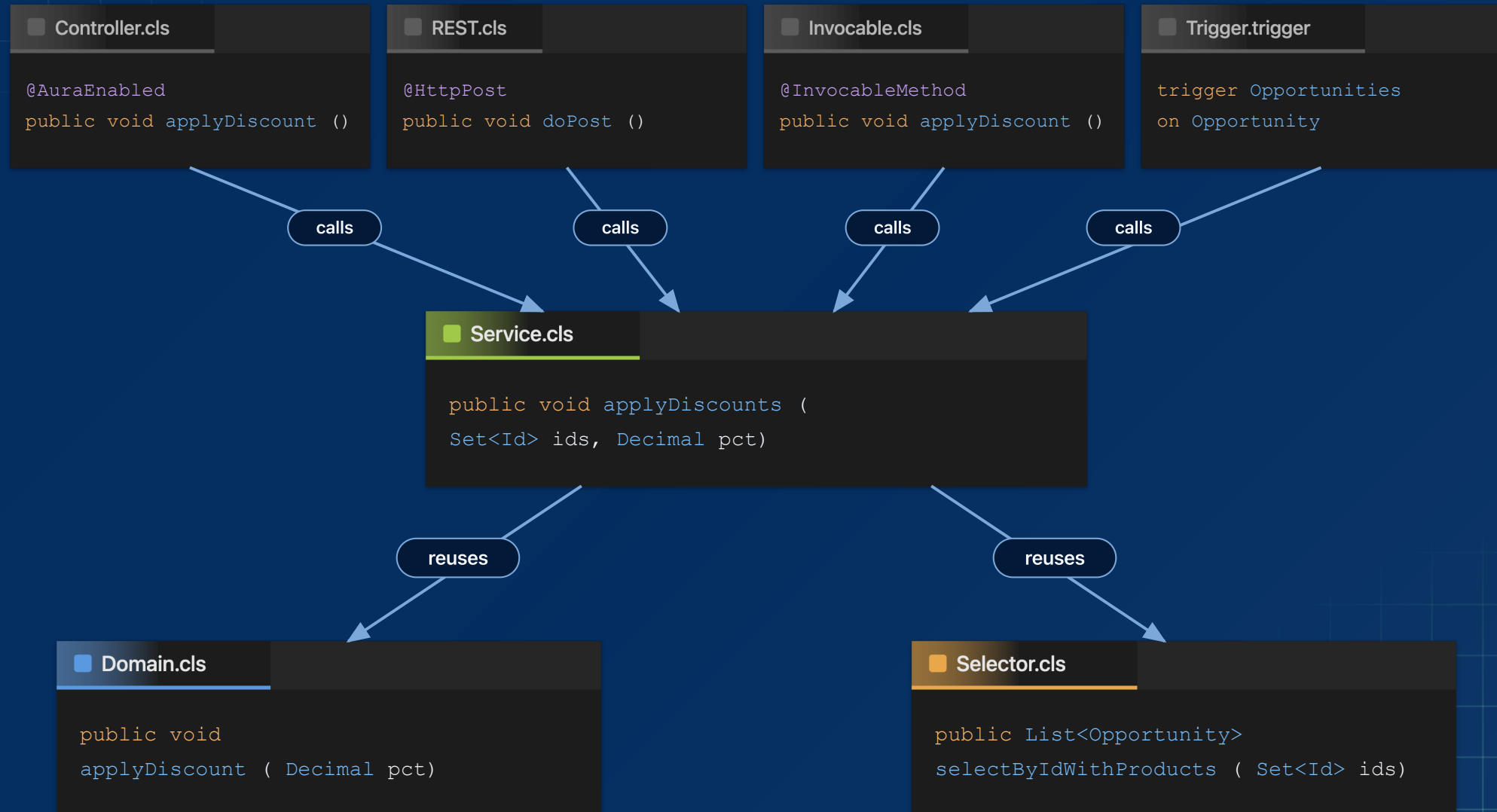
- Client-agnostic inputs and outputs
 -  `applyDiscounts(Set<Id> opportunityIds, Decimal discountPercentage)`
 -  `applyDiscount(ApplyInputForm form)`
- Client-agnostic exceptions
 -  `throw OpportunitiesServiceException`
 -  `throw AuraHandledException`
- **Bulkification** — collections in and out; don't force callers to loop
 -  `createInvoices(Set<Id> opportunityIds)`
 -  `createInvoice(Opportunity id)`
- **Transactions** — rollback on error; avoid partial commits; optional outer txn
- **Security** — user mode; elevates to system internally as needed

Service - Code Example

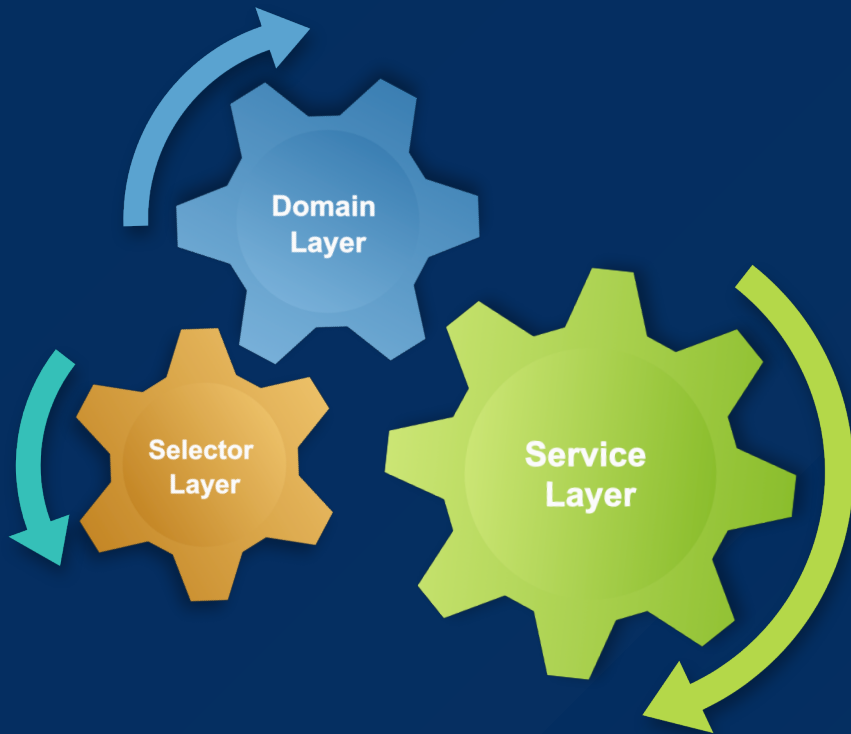
■ OpportunitiesService.cls

```
1 public void applyDiscounts(  
2     Set<Id> opportunityIds, Decimal discountPercentage) {  
3     Savepoint sp = Database.setSavepoint();  
4     try {  
5         Opportunities opportunities = Opportunities.newInstance(  
6             selector.selectByIdWithProducts(opportunityIds));  
7         opportunities.applyDiscount(discountPercentage);  
8         Database.update(opportunities.getRecords());  
9     } catch (Exception e) {  
10        Database.rollback(sp);  
11        throw new OpportunitiesServiceException(e.getMessage(), e);  
12    }  
13 }
```

Service - Consumers and Dependencies



Pattern Checklist



PATTERN CHECKLIST

Separation of Concerns



Service Layer



Domain Layer



Selector Layer



Domain ➔ Object Orientated Logic

- A Domain class combines **data and behavior** of an object
 - Opportunities — `Opportunities.newInstance(List<Opportunity> opportunities)`
 - Accounts — `Accounts.newInstance(List<Account> accounts)`
 - Base domain classes include common behaviors — `AbstractChargeable`
 - Top level domain classes extend — `TrainingWorkItems.cls` , `DeveloperWorkItems.cls`
- **Methods** are ways to interact with the object's behaviors,
`opportunities.applyDiscount` , `workItem.updateCostOfHoursWorked`
- Additional **methods** can respond to data manipulation scenarios (Triggers),
`opportunities.onBeforeUpdate` , `opportunities.onValidate`
 - Or can be split into a **sidecar handler** class if you prefer,
`new OpportunitiesTriggerHandler().onBeforeUpdate`

Apex Triggers - Poor Home for Logic

- No **methods** – only anonymous blocks and context variables
- Hard to **unit test** without DML and trigger execution
- No **mocking** – cannot swap dependencies
- Temptation to put SOQL, DML, and logic in one file
- **Pattern** – Apex Trigger delegates to **handler** or **domain** class

Opportunities.trigger

```
1 trigger Opportunities on Opportunity (  
2     before insert, before update, after insert) {  
3     OpportunitiesTriggerHandler domain =  
4         new OpportunitiesTriggerHandler();  
5     if (Trigger.isBefore && Trigger.isInsert) {  
6         domain.onBeforeInsert();  
7     } else if (Trigger.isBefore && Trigger.isUpdate) {  
8         domain.onBeforeUpdate(Trigger.oldMap);  
9     } else if (Trigger.isAfter && Trigger.isInsert) {  
10        domain.onAfterInsert();  
11    }  
12 }
```

Opportunities.trigger

```
1 trigger Opportunities on Opportunity (  
2     before insert, before update, after insert) {  
3     new MyTriggerHandlerFramework()  
4         .handleTrigger(Opportunities.class);  
5 }
```

Later in this series we will explore

`fflib_SObjectDomain.triggerHandler`

Domain and Trigger Handler Classes

Traditionally Domain classes have included methods handling trigger logic as well as other object behaviors in one class. The option exists to split CRUD logic into separate sidecar handlers if designed.

Service Layer → Domain

■ Opportunities.cls

```
1 public class Opportunities {
2
3     public void applyDiscount(
4         Decimal discountPercentage)
5     public void generate(
6         InvoicingService.InvoiceFactory invoiceFactory)
7     public static Decimal calculateDiscountFactor(
8         Decimal discountPercentage)
9     public Set<Id> getAccountIds()
10
11     // Optional inclusion in Domain –
12     // otherwise included in trigger handler
13
14     public void onBeforeInsert()
15     public void onBeforeUpdate(
16         Map<Id, SObject> existingRecords)
17     public void onAfterInsert()
18 }
```

Apex Trigger → Trigger Handler → Domain

■ OpportunitiesTriggerHandler.cls

```
1 public class OpportunitiesTriggerHandler {
2
3     public void onBeforeInsert()
4     public void onBeforeUpdate(
5         Map<Id, SObject> existingRecords)
6     public void onAfterInsert()
7 }
```

Apex Triggers by default run in **system mode** (even in API 67.0). Typically you would want any SOQL or DML in these methods (`onBeforeInsert` , `onBeforeUpdate` , `onAfterInsert`) to run as system mode unless you are prepared to grant all users object access – even if they don't use the app or objects directly.

Domain Checklist

- Record collections — wrap many records, not one
 -  `opportunities.newInstance(List<Opportunity> opportunities)`
 -  `opportunities.newInstance(Opportunity opportunity)`
- In-memory behavior — mutate records; no DML
 -  `void applyDiscount(Decimal discountPercentage)`
 -  `void applyDiscount() { update records; }`
- **Records encapsulated** — don't have callers pass them in
 -  `opportunities.applyDiscount(10)`
 -  `applyDiscount(Opportunity opportunity)`
- **Trigger logic** — Optional trigger handler sidecar class
- **Object-oriented** — data and behavior stay together
- **Security** — SOQL and DML are user mode, unless Apex Trigger context

Domain - Code Example #1

■ Opportunities.cls

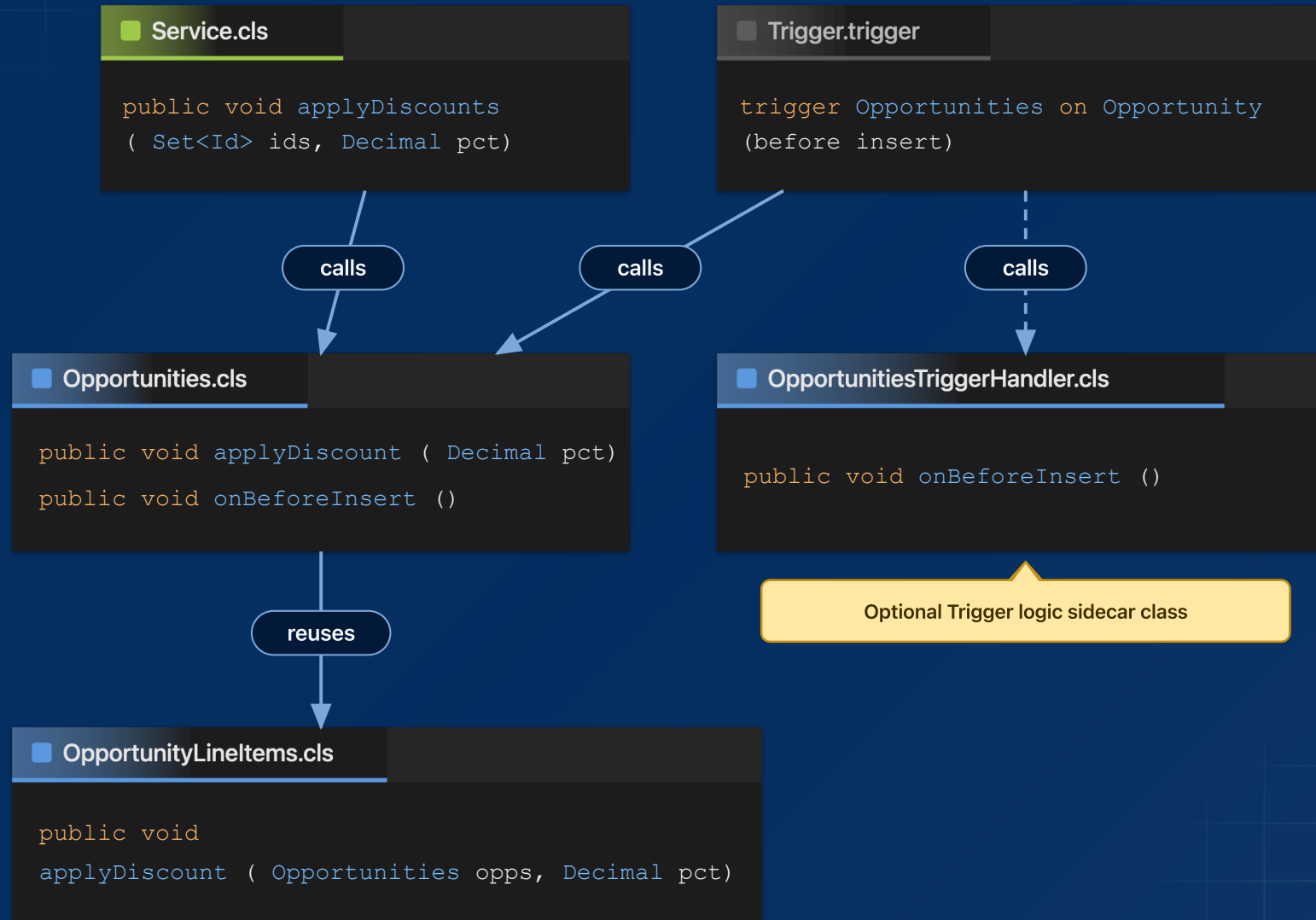
```
1 public void applyDiscount(Decimal discountPercentage) {
2     Decimal factor = calculateDiscountFactor(discountPercentage);
3     List<OpportunityLineItem> lines =
4         new List<OpportunityLineItem>();
5     for (Opportunity opportunity : records) {
6         if (opportunity.OpportunityLineItems == null ||
7             opportunity.OpportunityLineItems.isEmpty()) {
8             opportunity.Amount = opportunity.Amount * factor;
9         } else {
10             lines.addAll(opportunity.OpportunityLineItems);
11         }
12     }
13     OpportunityLineItems.newInstance(lines)
14         .applyDiscount(this, discountPercentage);
15 }
```

Domain - Code Example #2

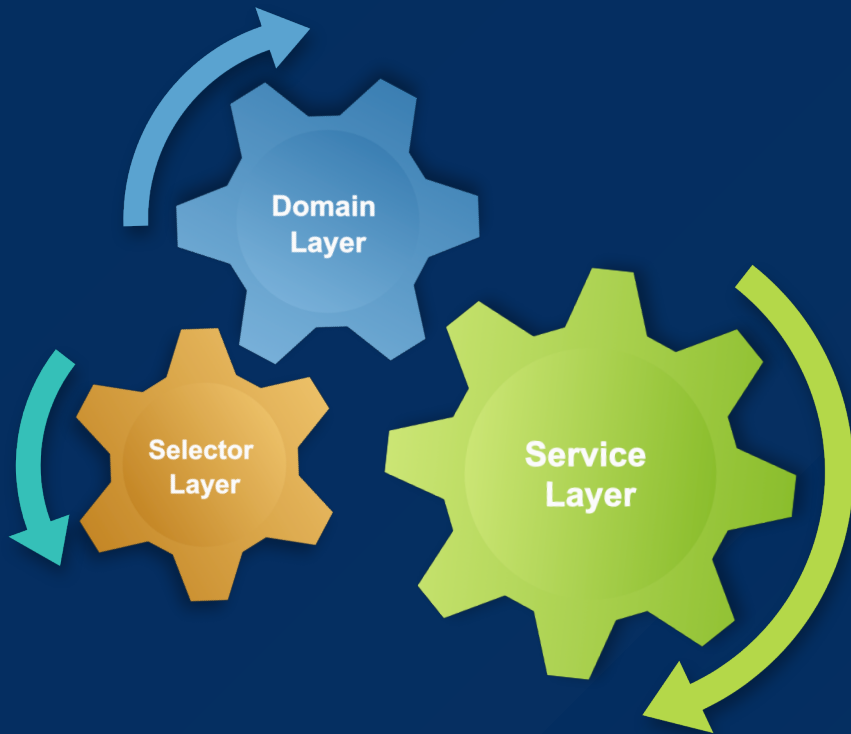
■ OpportunitiesTriggerHandler.cls

```
1 public void onValidate() {  
2     for (Opportunity opp : records) {  
3         if (opp.Type != null &&  
4             opp.Type.startsWith('Existing') &&  
5             opp.AccountId == null) {  
6             opp.AccountId.addError(  
7                 'You must provide an Account for existing Customers.');8         }  
9     }  
10 }
```

Domain / Trigger Handler - Consumers and Dependencies



Pattern Checklist



PATTERN CHECKLIST

Separation of Concerns



Service Layer



Domain Layer



Selector Layer



Selector → SOQL is Logic

- A Selector class encapsulates **query complexity** and supports **reuse**
- **Discovery** — required query easy to discover with named methods, `readyToInvoiceAsQueryLocator` , `selectRecentlyUsed`
- **Consistency** — apply same fields, ordering unless overridden
 - Start with common fields, add as needed
- Typically **class per object** — `OpportunitiesSelector` , `AccountsSelector`
- Consider **class per group of queries** — `WarehouseHandlingSelector`
- **Reuse** across Service, Domain, handlers, and sometimes controllers

Selector Checklist

- Provide result ordering, and field consistency by default
- Name **what** is returned and **how** it is filtered
 - `selectByIdWithProducts` , `selectByOpportunity` , `readyToInvoiceAsQueryLocator`
- Methods can also **dereference relationships** for callers — e.g.
`accountsSelector.selectByOpportunity(opportunities)`
- Elevation to system only when required, with overload, e.g.
 - `selectById(Set<Id> ids)`
 - `selectById(Set<Id> ids, Boolean systemMode)`
- Consider flattened data types to simplify consumption e.g.
`List<OpportunitySummary> selectOpportunitySummary(Set<Id> idSet)`
- Selectors run in user mode by default

Selector - Code Example

OpportunitiesSelector.selectById

```
1 public List<Opportunity> selectById(Set<Id> idSet) {
2     return [
3         SELECT Id, AccountId, Amount, CloseDate, Description,
4             DiscountType__c, Name, StageName, InvoicedStatus__c
5         FROM Opportunity
6         WHERE Id IN :idSet
7         ORDER BY Name
8     ];
9 }
```

OpportunitiesSelector.readyToInvoiceAsQueryLocator

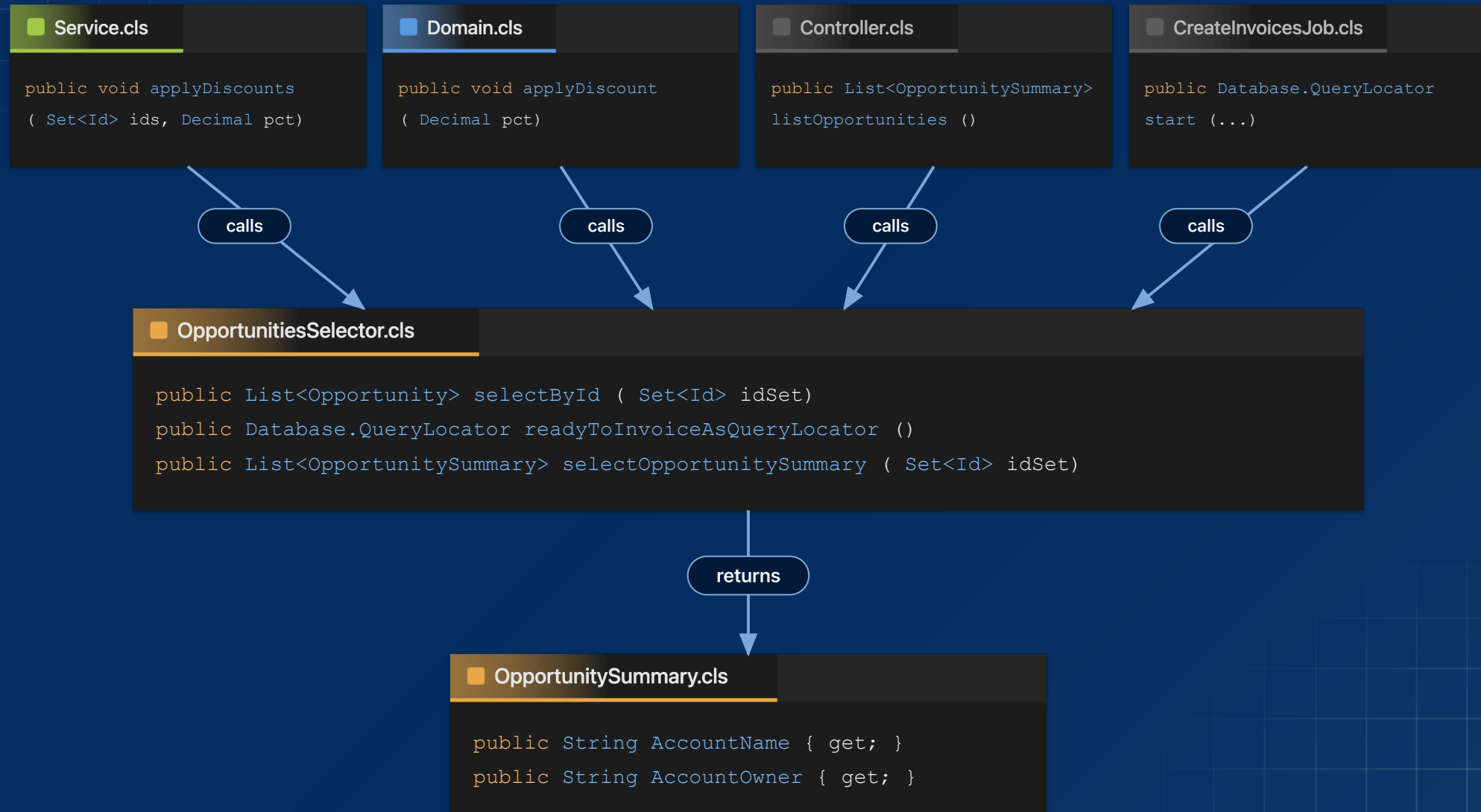
```
1 public Database.QueryLocator readyToInvoiceAsQueryLocator() {
2     return Database.getQueryLocator([
3         SELECT Id, AccountId, Amount, CloseDate, Description,
4             DiscountType__c, Name, StageName, InvoicedStatus__c
5         FROM Opportunity
6         WHERE InvoicedStatus__c = 'Ready'
7         ORDER BY Name
8     ]);
9 }
```

OpportunitiesSelector.selectOpportunitySummary

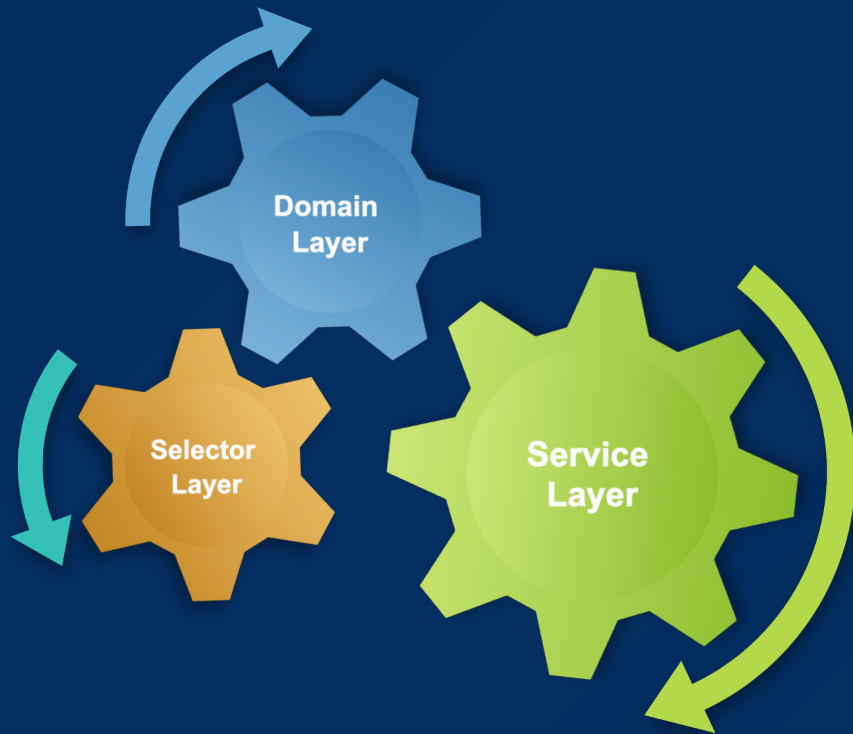
```
1 public List<OpportunitySummary> selectOpportunitySummary(Set<Id> idSet) {
2     List<OpportunitySummary> summaries = new List<OpportunitySummary>();
3     for (Opportunity opportunity : [
4         SELECT Id, AccountId, Amount, CloseDate, Description,
5             DiscountType__c, Name, StageName, InvoicedStatus__c,
6             Account.Name, Account.AccountNumber, Account.Owner.Name
7         FROM Opportunity
8         WHERE Id IN :idSet
9         ORDER BY Name
10    ]) {
11         summaries.add(new OpportunitySummary(opportunity));
12     }
13     return summaries;
14 }
```

① These examples are absent any kind of SOQL querying library which can help manage consistency between methods, such as common fields and sorting. The `fflib_SObjectSelector` base class provides support for this.

Selector - Consumers and Dependencies



Pattern Checklist



PATTERN CHECKLIST

Separation of Concerns



Service Layer



Domain Layer



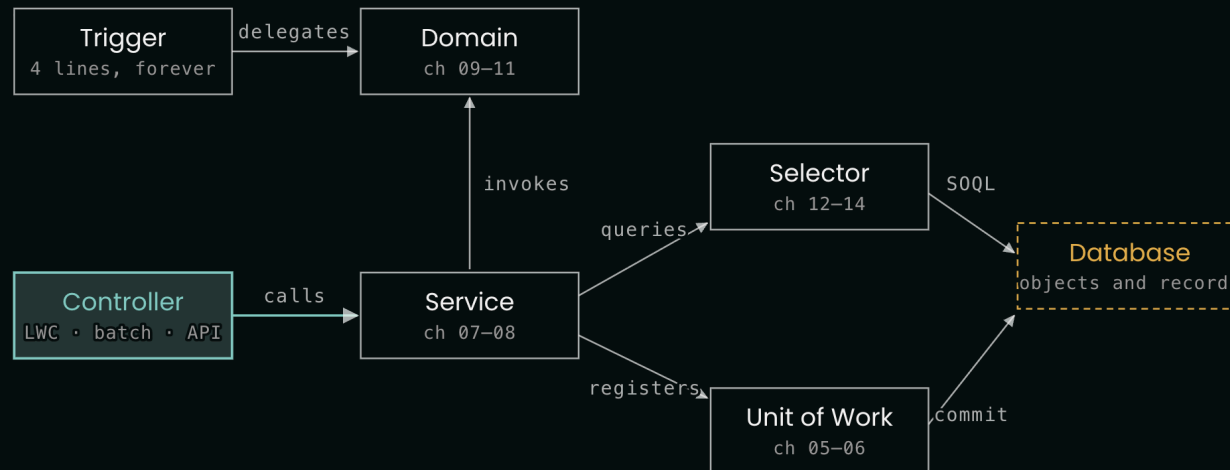
Selector Layer



Coming soon - FFLib Docs

HOW THE SEPARATION OF CONCERNS LAYERS WORK TOGETHER

SHOWING CONTROLLER



ENTRY POINT

Controller

Controller logic is now separate from your business logic. Your LWCs, batch jobs, queueables and REST endpoints now all delegate business logic to the service layer.

THE RULE

Controllers call the Service and nothing else.

CHAPTERS

07	The Service Layer	24:08
08	Implementing the Service Layer	35:22

START AT CHAPTER 07

What's next in the series

SESSION



Session #1 - Separation of Concerns in Apex: Why Your Future Self Will Thank You



Session #2 - Service Layers Explained: Coordinating Business Logic in Apex



Session #3 - Domain vs Service: Where Should Your Apex Logic Live?



Session #4 - Query Logic as a First-Class Architecture Concern



Session #5 - Mocking in Apex: Why It Changes Everything



Session #6 - Enterprise-Scale Apex Across Multiple Packages

Thank you!

Andrew Fawcett · Code With Sally

FFLib Series · Session 001

